

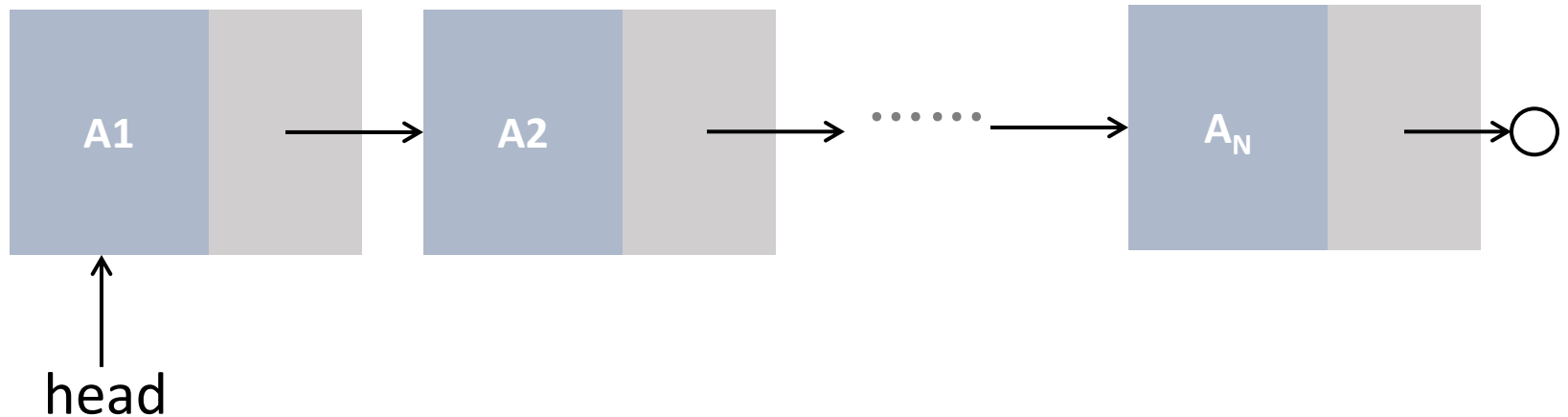
Data Structures and Algorithms

Linked Lists - Side Notes

Data Science

Beijing Normal-Hong Kong Baptist University

Node



- In linked list, the only way to locate the next node is from its predecessor
 - Every node stores the address of the next node

Node

```
typedef struct node{  
    double data;  
    struct node* next;  
} Node;
```

```
struct node{  
    double data;  
    struct node* next;  
};  
typedef struct node Node;
```

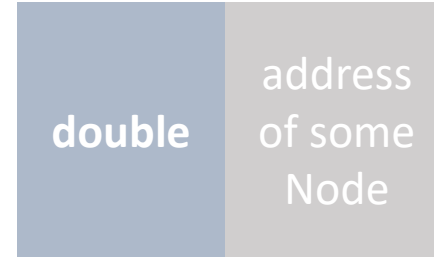
- Define a new struct called **node**
 - struct in C is a composite user-defined data type that holds data items of different kinds
- Give this type a new name **Node**
 - Node n1; // declare a Node type variable

Node

```
#include<stdio.h>
#include<stdlib.h>
typedef struct node{
    double data;
    struct node* next;
} Node;
int main(){
    Node n1;
    n1.data = 0.1;
    n1.next = NULL;

    Node* n2 = (Node*)malloc(sizeof(Node));
    n2->data = 0.2;
    n2->next = &n1;

    printf("n1.data: %f, n1.next:%x\nn2->data:%f, n2->next:
           %x\n",n1.data, n1.next, n2->data, n2->next );
}
```



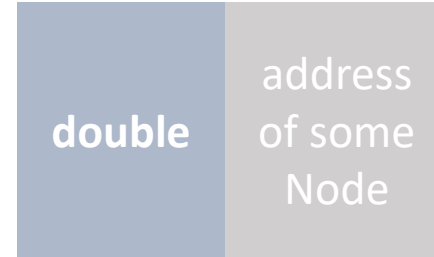
- What's the output?

Node

```
#include<stdio.h>
#include<stdlib.h>
typedef struct node{
    double data;
    struct node* next;
} Node;
int main(){
    Node n1;
    n1.data = 0.1;
    n1.next = NULL;

    Node* n2 = (Node*)malloc(sizeof(Node));
    n2->data = 0.2;
    n2->next = &n1;

    printf("n1.data: %f, n1.next:%x\nn2->data:%f, n2->next:
           %x\n",n1.data, n1.next, n2->data, n2->next );
}
```



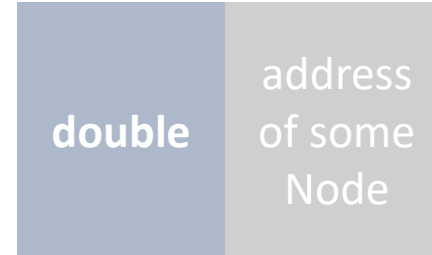
- What's the output?

Node

```
#include<stdio.h>
#include<stdlib.h>
typedef struct node{
    double data;
    struct node* next;
} Node;
int main(){
    Node n1;
    n1.data = 0.1;
    n1.next = NULL;

    Node* n2 = (Node*)malloc(sizeof(Node));
    n2->data = 0.2;
    n2->next = &n1;

    printf("n1.data: %f, n1.next:%x\nn2->data:%f, n2->next:
        %x\n",n1.data, n1.next, n2->data, n2->next );
}
```



- What's the output?

```
n1.data: 0.100000, n1.next:0
n2->data:0.200000, n2->next:61ff18
```

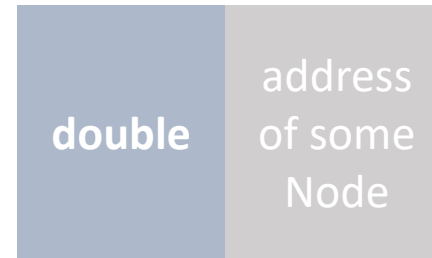
```

#include<stdio.h>
#include<stdlib.h>
typedef struct node{
    double data;
    struct node* next;
}Node;
int main(){
    Node n1;
    n1.data = 0.1;
    n1.next = NULL;

    Node* n2 = (Node*)malloc(sizeof(Node));
    n2->data = 0.2;
    n2->next = &n1;

    Node* n3 = (Node*)malloc(sizeof(Node));
    n3->data = 0.3;
    n3->next = n2;
    printf("n1.data:%f, n1.next:%p\nn2->data:%f,n2->next:%p\nn3->data:%f, n3->next:%p\n",n1.data, n1.next, n2->data, n2->next, n3->data, n3->next );
    printf("The address of n1:%p,n2:%p,n3:%p\n", &n1,n2,n3);
}

```



• What's the output?

```
#include<stdio.h>
#include<stdlib.h>
typedef struct node{
    double data;
    struct node* next;
}Node;
```

```
int main(){
    Node n1;
    n1.data = 0.1;
    n1.next = NULL;
```

```
    Node* n2 = (Node*)malloc(sizeof(Node));
    n2->data = 0.2;
    n2->next = &n1;
```

```
    Node* n3 = (Node*)malloc(sizeof(Node));
    n3->data = 0.3;
    n3->next =
```

```
    printf("n1.
>data:%f, n3->r
n3->data, n3->r
    printf("The
}
```

double

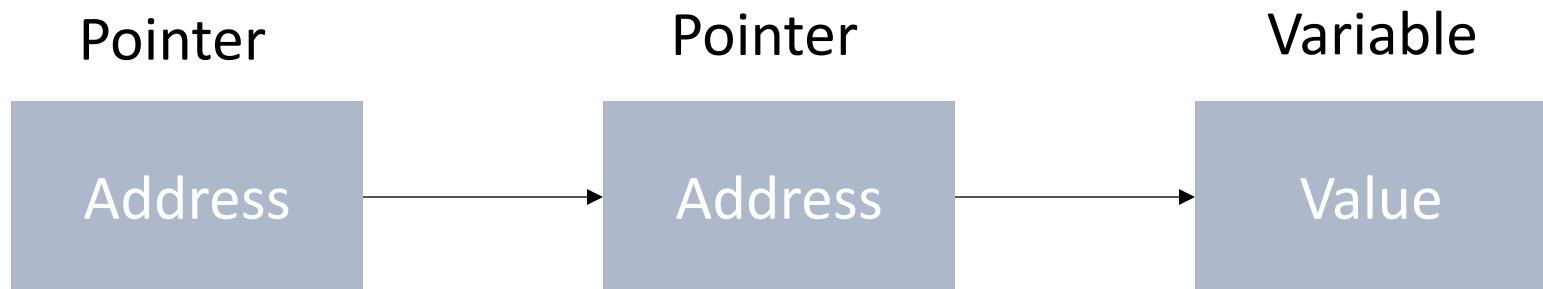
address
of some
Node

n1.data: 0.100000, n1.next:0
n2->data:0.200000, n2->next:61ff18
n3->data:0.300000, n3->next:691668:
The address of n1:61ff18,n2:691668,n3:691680

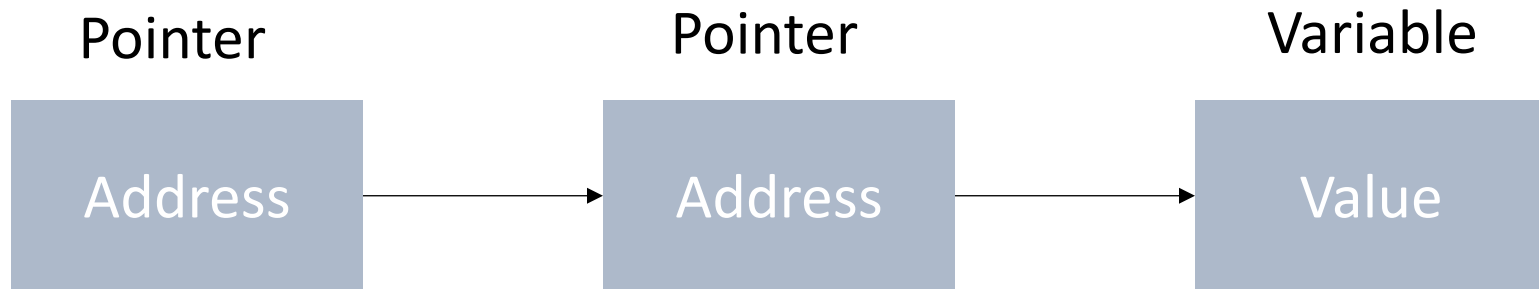
• What's the output?

Pointer to Pointer

- When we define a pointer to a pointer, the **first pointer** contains the **address** of the **second pointer**, which points to the location that contains the actual value as shown below.



Pointer to Pointer



```
Node* head = (Node*) malloc(sizeof(Node));  
head->data = 0.2;  
head->next = NULL;  
  
Node** phead = &head;  
  
Node newHead;  
newHead.data = 0.1;  
newHead.next = head;  
  
*phead = &newHead;
```

Pointer to Pointer

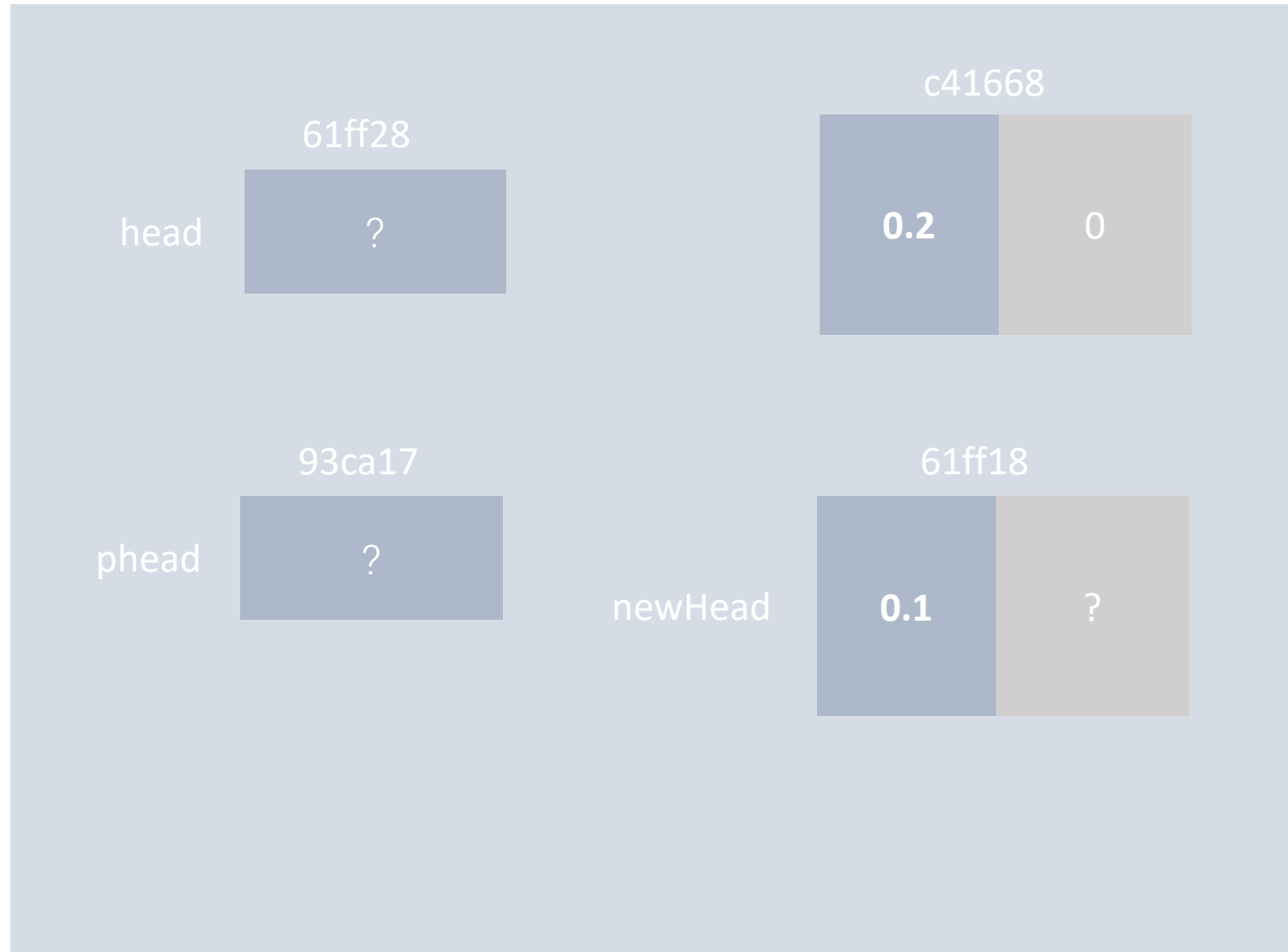
```
int main() {
    Node* head = (Node*)malloc(sizeof(Node));
    head->data = 0.2;
    head->next = NULL;

    Node** phead = &head;
    printf("The value of phead:%p\n", phead);
    printf("The value of head:%p\n", head);
    printf("The value of node which head points to:%f\n", head->data);

    Node newHead;
    newHead.data = 0.1;
    newHead.next = head;

    *phead = &newHead;

    printf("The value of phead:%p\n", phead);
    printf("The value of head:%p\n", head);
    printf("The value of node which head points to:%f", head->data);}
```



Pointer to Pointer

```
int main() {
    Node* head = (Node*)malloc(sizeof(Node));
    head->data = 0.2;
    head->next = NULL;

    Node** phead = &head;
    printf("The value of phead:%p\n", phead);
    printf("The value of head:%p\n", head);
    printf("The value of node which head points to:%f\n", head->data);

    Node newHead;
    newHead.data = 0.1;
    newHead.next = head;

    *phead = &newHead;

    printf("The value of phead:%p\n", phead);
    printf("The value of head:%p\n", head);
    printf("The value of node which head points to:%f\n", head->data);
}
```

The value of phead:61ff28
The value of head:c41668
The value of node which head points to:0.200000
The value of phead:61ff28
The value of head:61ff18
The value of node which head points to:0.100000

Memory allocation and free

- Two ways for memory allocation
 - static memory allocation
 - Memory is allocated invisibly when a variable is declared, e.g., `int i`, `Node* n1`, etc.
 - dynamic memory allocation
 - Memory is allocated visibly by calling some **allocation functions**
- Dynamic memory is allocated in the **heap** (also called the free storage, a large pool of unused memory area) by the system
- Static memory is allocated in the **stack** (last-in-first-out) by the system

Memory allocation and free

- Two basic functions for dynamic allocation:
 - `void *malloc(long size)`
 - Allocate a block of memory of specified size and returns a pointer of type void
 - We can assign it to a pointer of any type
 - A null pointer is returned if there is not enough space in the heap
 - `void free(void *ptr)`
 - releases the used memory when it is no longer needed

```

int main() {
    Node n1;
    n1.data = 0.1;
    n1.next = NULL;

    //Pointer is allocated on the stack
    // the node object it is pointing to is allocated on the heap

    Node* n2 = (Node*)malloc(sizeof(Node));
    n2->data = 0.2;
    n2->next = &n1;
    Node* n3 = (Node*)malloc(sizeof(Node));
    n3->data = 0.3;
    n3->next = n2;

    if (n3 != NULL) {
        printf("n3:%p\n", n3->next);
        free(n3);
        n3 = NULL;
        //printf("n3 :%p", n3->next); // NULL pointer
    }
}

```